

Curso Profesional de JasperReports 6.20.0 Community

Módulo 6 — Exportación

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Módulo 6 — Teoría de exportación

Proyecto acumulativo: **EditorialReports**. Fuente original preservada en `.github/source/M6_ORIGINAL.md`.

La teoría conserva los objetivos del material original y corrige las APIs que no corresponden a JasperReports Library 6.20.0. Cada corrección se traza a un checkpoint compilado y ejecutado.

Punto 6.1 — Exportación a PDF

Objetivos de aprendizaje

- Comprender el papel del exportador PDF y su ubicación en la biblioteca.
- Diferenciar los métodos simples de JasperExportManager de los exportadores avanzados.
- Configurar las propiedades del exportador mediante SimplePdfExporterConfiguration.
- Establecer metadatos del documento PDF (título, autor, palabras clave).
- Aplicar protección con contraseña y permisos al PDF generado.
- Documentar la exportación a PDF del proyecto EditorialReports.

Bloque 1 — De JasperPrint al PDF: exportación simple y avanzada

El punto de partida no cambia respecto a M5: informe_ventas.jrxml se compila, se llena con SQLite y produce un JasperPrint de seis páginas. El Módulo 6 comienza después del llenado. Esto es importante porque el formato PDF no se diseña en un JRXML diferente: el mismo documento en memoria puede enviarse a varios exportadores. JasperReports ofrece una vía simple, JasperExportManager.exportReportToPdfFile, y una vía avanzada basada en JRPdfExporter. La primera es suficiente cuando basta con escribir un PDF sin personalización; la segunda separa entrada, salida y configuración y permite controlar metadatos, compresión y seguridad.

JAVA

```
JRPdfExporter exportador = new JRPdfExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(rutaPdf));
exportador.exportReport();
```

SimpleExporterInput adapta el JasperPrint a la entrada esperada por el exportador. SimpleOutputStreamExporterOutput define el destino binario y exportReport() realiza la conversión. El checkpoint 6.1 conserva también la compilación del subreporte antes del maestro, la conexión heredada y todos los parámetros de M5; sólo sustituye la exportación simple de ventas por la ruta avanzada.

El E2E verifica que el archivo principal empieza por %PDF-, que JasperPrint sigue teniendo seis páginas y que todos los informes heredados continúan generándose. Por tanto, el cambio está integrado en el proyecto acumulativo completo.

Bloque 2 — Metadatos correctos con SimplePdfExporterConfiguration

SimplePdfExporterConfiguration configura propiedades que afectan al documento PDF completo. En JasperReports 6.20.0 los metadatos se establecen con setMetadataTitle, setMetadataAuthor, setMetadataSubject, setMetadataKeywords y setMetadataCreator. El material original empleaba nombres genéricos como setTitle y setAuthor; el checkpoint corrige esa diferencia sin alterar el objetivo pedagógico.

JAVA

```
SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
configuracion.setMetadataTitle("Informe de Ventas - EditorialReports");
configuracion.setMetadataAuthor("Departamento Comercial");
configuracion.setMetadataSubject("Resumen de ventas del catálogo");
configuracion.setMetadataKeywords("ventas, catálogo, libros, editorial");
configuracion.setMetadataCreator("JasperReports 6.20.0");
configuracion.setDisplayMetadataTitle(Boolean.TRUE);
configuracion.setCompressed(Boolean.TRUE);
exportador.setConfiguration(configuracion);
```

Los metadatos son propiedades del fichero, no elementos visuales de la página. Por eso no modifican bandas, estilos ni el número de páginas. setDisplayMetadataTitle(Boolean.TRUE) permite que los lectores que respetan esa preferencia utilicen el título documental. setCompressed(Boolean.TRUE) activa la compresión del PDF.

La fuente original proponía aplicar setCharacterEncoding directamente sobre la configuración PDF. Ese método no forma parte de SimplePdfExporterConfiguration en esta versión y no se reproduce. La corrección queda probada por la compilación Maven y por pdfinfo, que comprueba título, autor y creador en el archivo generado.

Bloque 3 — Cifrado, contraseñas y permisos

Para que el alumno pueda comprobar simultáneamente un PDF normal con metadatos y otro protegido, el checkpoint genera informe_ventas.pdf e informe_ventas_protegido.pdf. El segundo activa cifrado y separa contraseña de usuario y de propietario.

JAVA

```
SimplePdfExporterConfiguration configuracion = new SimplePdfExporterConfiguration();
configuracion.setEncrypted(Boolean.TRUE);
configuracion.setUserPassword("editorial2026");
configuracion.setOwnerPassword("editorial-admin");
configuracion.setAllowedPermissionsHint("PRINTING|COPY|SCREENREADERS");
```

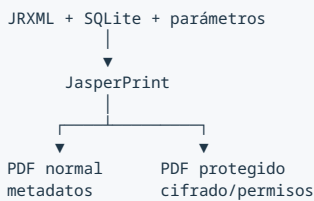
`setUserPassword` define la contraseña necesaria para abrir el documento. `setOwnerPassword` protege las operaciones reservadas al propietario. `setAllowedPermissionsHint` expresa los permisos concedidos. La fuente proponía `setPdfPassword(usuario, propietario)` y un `EnumSet<PdfPermissionsEnum>`; la práctica conserva el objetivo de seguridad pero usa la API que realmente compila en 6.20.0.

El E2E ejecuta `pdftinfo -upw editorial2026` sobre el PDF protegido. Si la contraseña descrita por la práctica no abre el fichero, el checkpoint falla. Esto vincula la explicación directamente al comportamiento producido.

Bloque 4 — Entrada, salida y reutilización del mismo JasperPrint

El exportador avanzado sigue el patrón común de JasperReports: una entrada, una salida y una configuración. El `JasperPrint` no se vuelve a llenar para cada formato. Esto permite que, a partir de 6.2, el mismo documento se entregue también a XLSX, HTML, CSV, XML y RTF.

TEXT



Separar llenado y exportación reduce trabajo y mantiene coherencia: todos los formatos representan exactamente el mismo estado de datos. Si `JasperFillManager.fillReport` falla, ninguno de los exportadores se ejecuta. Si sólo falla `JRPdfExporter`, el problema está en la configuración o en la salida PDF.

La ruta SQLite se mantiene como `jdbc:sqlite:../EditorialReportsJava/data/editorial.db` porque el generador se ejecuta desde `EditorialReports`. También se mantiene `System.exit(1)` en el catch para que un fallo sea visible en CI.

Bloque 5 — Evidencia reproducible y documentación

Una práctica de exportación sólo está terminada cuando el archivo generado puede inspeccionarse. El checkpoint 6.1 comprueba compilación Java, compilación JRXML, llenado del documento y validación del PDF. A ello se añaden los contratos heredados: 14 libros, 9 ventas, 31 unidades y 633,40 €.

TEXT

```
M5/5.6
├── M6/6.1
│   ├── mismo informe_ventas.jrxml
│   ├── mismo EditorialStyles.jrtx
│   ├── GeneradorInformeVentas.java modificado
│   ├── EXPORTACION_PDF.md nuevo
│   ├── output/informe_ventas.pdf
│   └── output/informe_ventas_protegido.pdf
```

La auditoría de trazabilidad exige que ningún archivo heredado del proyecto de informes desaparezca o cambie. M6 altera la capa Java de exportación y añade documentación/salidas, pero no reescribe el diseño validado de M5.

`EXPORTACION_PDF.md` registra las APIs reales y los archivos generados. El workflow E2E produce además un artefacto runtime. Así, teoría, práctica, fuente Java y evidencia de ejecución describen el mismo mecanismo.

Punto 6.2 — Exportación a Excel

Objetivos de aprendizaje

- Comprender las diferencias entre los formatos XLS y XLSX.
- Utilizar el exportador JRXLsxExporter para generar archivos Excel modernos.
- Configurar las propiedades del exportador mediante SimpleXlsxExporterConfiguration.
- Ajustar el nombre de la hoja, el ancho de columnas y las celdas combinadas.
- Aplicar formato a las celdas exportadas.
- Documentar la exportación a Excel del proyecto EditorialReports.

Bloque 1 — XLS frente a XLSX y dependencia de Apache POI

El segundo punto añade Excel al mismo JasperPrint que ya se exporta a PDF. XLS es el formato binario histórico de Excel; XLSX es el formato OOXML moderno, empaquetado como ZIP y compuesto por documentos XML. EditorialReports utiliza JRXLsxExporter porque el objetivo es producir `output/informe_ventas.xlsx`.

JasperReports 6.20.0 declara Apache POI como dependencia opcional. Por eso un proyecto Maven que sólo había generado PDF puede compilar JasperReports y, sin embargo, fallar en tiempo de ejecución al usar XLSX si POI no está en el classpath. El checkpoint 6.2 añade explícitamente `poi` y `poi-ooxml` 5.1.0 al `pom.xml`.

XML

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.1.0</version>
</dependency>
```

El E2E comprueba que Maven resuelve las dependencias, que el archivo empieza por la firma PK propia de ZIP y que el paquete OOXML no contiene entradas corruptas. Así la dependencia no queda como una nota teórica: forma parte del contrato ejecutable.

Bloque 2 — Dos niveles de configuración XLSX

La fuente original concentraba nombre de hoja, cuadrícula, bloqueo y paleta en `SimpleXlsxExporterConfiguration`. En la API real esas opciones se dividen. `SimpleXlsxReportConfiguration` controla cómo un `JasperPrint` se transforma en hojas y celdas; `SimpleXlsxExporterConfiguration` contiene opciones del libro/exportador.

JAVA

```
SimpleXlsxReportConfiguration informe = new SimpleXlsxReportConfiguration();
informe.setSheetNames(new String[]{"Ventas"});
informe.setShowGridLines(Boolean.FALSE);
informe.setCellLocked(Boolean.FALSE);
informe.setCellHidden(Boolean.FALSE);
informe.setDetectCellType(Boolean.TRUE);
informe.setOnePagePerSheet(Boolean.FALSE);

SimpleXlsxExporterConfiguration libro = new SimpleXlsxExporterConfiguration();
libro.setCreateCustomPalette(Boolean.TRUE);
```

Separar ambas configuraciones evita llamar métodos sobre una clase que no los define. `setDetectCellType` intenta conservar valores numéricos como tipos Excel en lugar de convertir todo a texto. `setOnePagePerSheet(FALSE)` evita crear una hoja por cada página del `JasperPrint`.

Bloque 3 — Flujo de JRXLsxExporter

JRXLsxExporter recibe el mismo documento ya utilizado por PDF. Se aplican las dos configuraciones, se establece `SimpleExporterInput` y la salida binaria con `SimpleOutputStreamExporterOutput`.

JAVA

```
JRXLsxExporter exportador = new JRXLsxExporter();
exportador.setConfiguration(informe);
exportador.setConfiguration(libro);
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleOutputStreamExporterOutput(rutaXlsx));
exportador.exportReport();
```

JasperReports convierte coordenadas y elementos gráficos a una rejilla de celdas. Esa traducción nunca es idéntica a un PDF porque Excel trabaja con filas y columnas, pero conserva información útil de texto, números y estilos. Cuanto más tabular sea el informe, más natural será el resultado.

El diseño JRXML no cambia. La auditoría compara byte a byte `informe_ventas.jrxml` y `EditorialStyles.jrtx` con M5/5.6. El objetivo del punto es el exportador, no rediseñar el informe para Excel.

Bloque 4 — Nombre de hoja, cuadrícula, tipos y edición

El nombre Ventas se aplica mediante `setSheetNames`. El workflow abre internamente `xl/workbook.xml` del XLSX y exige que exista una hoja con ese nombre. Esto convierte un requisito de la práctica en una prueba automática.

`setShowGridLines(FALSE)` elimina las líneas de cuadrícula predeterminadas. `setCellLocked(FALSE)` y `setCellHidden(FALSE)` dejan las celdas sin protección adicional. `setCreateCustomPalette(TRUE)` intenta conservar los colores del informe dentro de las limitaciones del formato/exportador.

TEXT

```

JasperPrint (6 páginas)
├── JRXLsxExporter
│   ├── ReportConfiguration
│   │   ├── sheet = Ventas
│   │   ├── gridlines = false
│   │   └── detectCellType = true
│   └── ExporterConfiguration
│       └── custom palette = true
└── informe_ventas.xlsx

```

La práctica no afirma que cada píxel del PDF tenga una equivalencia exacta en Excel. Enseña qué propiedades controla el exportador y cómo verificar el resultado estructural.

Bloque 5 — Validación OOXML y continuidad acumulativa

Un fichero con extensión `.xlsx` no es suficiente evidencia. El E2E comprueba tamaño mayor que cero, firma ZIP, integridad de todas las entradas y presencia de la hoja Ventas. A la vez, vuelve a ejecutar los cinco generadores heredados y comprueba los invariantes SQLite.

El checkpoint 6.2 es físicamente 6.1 más: dos dependencias Maven, la lógica XLSX en `GeneradorInformeVentas.java` y `EXPORTACION_EXCEL.md`. No elimina la exportación PDF ni el PDF protegido.

TEXT

```

6.1: PDF + PDF protegido
├── 6.2: PDF + PDF protegido + XLSX
│   └── hoja Ventas validada desde workbook.xml

```

Esta acumulación permite que los puntos posteriores reutilicen el mismo XLSX sin duplicar el llenado. La documentación distingue además la configuración por informe y la configuración por exportador para impedir que reaparezca el error técnico de la fuente original.

Punto 6.3 — Exportación a HTML

Objetivos de aprendizaje

- Comprender el papel del exportador HTML y sus limitaciones respecto a PDF.
- Configurar el exportador JRHtmlExporter con SimpleHtmlExporterConfiguration.
- Exportar las imágenes del informe a un directorio y referenciarlas desde el HTML.
- Añadir cabecera, pie y separador de páginas al archivo HTML.
- Integrar el HTML generado con una hoja de estilos CSS externa.
- Documentar la exportación a HTML del proyecto EditorialReports.

Bloque 1 — HTML como exportación navegable del JasperPrint

HTML representa el informe mediante marcado que un navegador puede interpretar. A diferencia del PDF, el resultado puede depender de recursos externos —imágenes y CSS—, por lo que una validación correcta debe considerar el archivo principal y esos recursos. JasperReports 6.20.0 utiliza `HtmlExporter`; el nombre `JRHtmlExporter` del material original no es la clase empleada por el checkpoint compilado.

JAVA

```
HtmlExporter exportador = new HtmlExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
```

El mismo JasperPrint de seis páginas alimenta el exportador. No se ejecuta de nuevo SQLite y no se modifica `informe_ventas.jrxml`. El checkpoint crea `output/informe_ventas.html`, `output/images/` y `output/styles/editorial.css`.

Bloque 2 — SimpleHtmlExporterConfiguration: cabecera, pie y páginas

`SimpleHtmlExporterConfiguration` controla elementos HTML generales. El checkpoint define una cabecera completa con UTF-8, título y enlace a CSS; define también el cierre del documento y un separador entre páginas del JasperPrint.

JAVA

```
SimpleHtmlExporterConfiguration configuracion = new SimpleHtmlExporterConfiguration();
configuracion.setHtmlHeader("<html><head><meta charset='UTF-8'>"
    + "<title>Informe de Ventas - EditorialReports</title>"
    + "<link rel='stylesheet' href='styles/editorial.css'>"
    + "</head><body>");
configuracion.setHtmlFooter("</body></html>");
configuracion.setBetweenPagesHtml("<hr class='salto-pagina' />");
```

La codificación se declara en el HTML y también en el objeto de salida. El enlace CSS es relativo al HTML final, por eso el archivo se copia a `output/styles`. El separador permite aplicar un estilo específico entre páginas sin introducir contenido en el JRXML.

Bloque 3 — SimpleHtmlExporterOutput y recursos de imagen

La fuente original asignaba rutas de imágenes mediante métodos de `SimpleHtmlExporterConfiguration`. En el modelo real, el tratamiento de recursos corresponde al output. El checkpoint crea `SimpleHtmlExporterOutput` y le asigna un `FileHtmlResourceHandler`.

JAVA

```
SimpleHtmlExporterOutput salida = new SimpleHtmlExporterOutput(rutaHtml, "UTF-8");
salida.setImageHandler(
    new FileHtmlResourceHandler(new File("output/images"), "images/{0}"));
exportador.setExporterOutput(salida);
```

El handler conoce dónde escribir físicamente los recursos y qué URI debe aparecer en el HTML. Así el navegador puede resolver `images/...` desde el archivo ubicado en `output`.

Aunque una ejecución concreta pueda necesitar cero o varias imágenes externas, la carpeta y la estrategia quedan preparadas. El E2E exige que el directorio exista y que el HTML sea coherente con la estructura documentada.

Bloque 4 — CSS externo y separación de presentación web

`editorial.css` no sustituye los estilos JasperReports. Los estilos JRTX siguen controlando el JasperPrint; el CSS añade presentación al contenedor HTML que rodea el contenido exportado.

CSS

```
body {
    margin: 24px;
    background: #ffffff;
    color: #173f6b;
    font-family: "DejaVu Sans", Arial, sans-serif;
}
.salto-pagina {
    border-top: 1px solid #d6eaf8;
}
```

Esta separación evita incrustar grandes cantidades de CSS en Java y hace visible en el árbol del proyecto el recurso que debe publicarse junto al HTML. La práctica visual enseña a crear el CSS, copiarlo y validar el enlace.

El E2E comprueba que el HTML contiene meta charset, título, referencia styles/editorial.css y etiqueta de cierre; también exige que el CSS exista en la salida.

Bloque 5 — Validación funcional del HTML

La validación se realiza en varias capas. Maven verifica que las clases y métodos existen. La ejecución verifica que el exportador serializa el JasperPrint real. Las aserciones estructurales comprueban las piezas mínimas del documento y sus recursos.

TEXT

```
JasperPrint
├── HtmlExporter
│   ├── SimpleHtmlExporterConfiguration
│   │   ├── header
│   │   ├── footer
│   │   └── betweenPagesHtml
│   └── SimpleHtmlExporterOutput
│       └── FileHtmlResourceHandler
└── output/
    ├── informe_ventas.html
    ├── images/
    └── styles/editorial.css
```

El resultado se acumula sobre 6.2: PDF, PDF protegido y XLSX siguen generándose en la misma ejecución. Esta continuidad está verificada por el workflow, no sólo descrita en el texto.

Punto 6.4 — Exportación a CSV y otros formatos

Objetivos de aprendizaje

- Comprender las características del formato CSV y sus limitaciones.
- Utilizar el exportador JRCSVExporter con SimpleCsvExporterConfiguration.
- Configurar el separador de campos y la codificación del archivo CSV.
- Exportar a formato XML con JRXMLExporter y a RTF con JRRtfExporter.
- Combinar varios exportadores en una misma ejecución del programa.
- Documentar la exportación a CSV, XML y RTF del proyecto EditorialReports.

Bloque 1 — CSV: exportación orientada a datos

CSV sacrifica la geometría de página para producir texto delimitado. JRCSVExporter recorre el contenido textual exportable del JasperPrint y genera registros. En EditorialReports se utiliza punto y coma como delimitador de campos y salto de línea como delimitador de registros.

JAVA

```
SimpleCsvExporterConfiguration configuracion = new SimpleCsvExporterConfiguration();
configuracion.setFieldDelimiter(";");
configuracion.setRecordDelimiter("\n");
configuracion.setWriteBOM(Boolean.TRUE);
```

La fuente original proponía `setEncoding("UTF-8")` sobre `SimpleCsvExporterConfiguration`. En el checkpoint real, la codificación pertenece al `ExporterOutput`:

JAVA

```
exportador.setExporterOutput(new SimpleWriterExporterOutput(rutaCsv, "UTF-8"));
```

El BOM facilita que aplicaciones de escritorio detecten UTF-8. El E2E comprueba los bytes EF BB BF, la presencia de punto y coma y que exista más de una línea. Por tanto, la prueba valida el fichero producido, no sólo la llamada Java.

Bloque 2 — XML: representación estructural del documento

JRXMLExporter serializa el JasperPrint a XML. No produce el XML de negocio original ni ejecuta una consulta diferente: representa el documento ya llenado. SimpleXmlExporterOutput permite definir archivo y codificación.

JAVA

```
JRXMLExporter exportador = new JRXMLExporter();
SimpleXmlExporterOutput salida = new SimpleXmlExporterOutput(rutaXml, "UTF-8");
salida.setEmbeddingImages(Boolean.TRUE);
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(salida);
exportador.exportReport();
```

Con `setEmbeddingImages(Boolean.TRUE)` los recursos gráficos pueden quedar embebidos según la representación del exportador, evitando referencias externas. La prueba automática exige que el fichero comience con una declaración XML.

Bloque 3 — RTF: salida para procesadores de texto

JRRtfExporter produce Rich Text Format. La salida es textual y el checkpoint utiliza `SimpleWriterExporterOutput(rutaRtf, "UTF-8")`.

JAVA

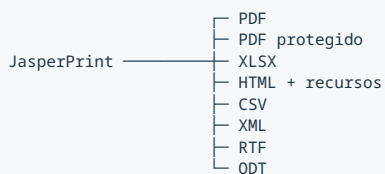
```
JRRtfExporter exportador = new JRRtfExporter();
exportador.setExporterInput(new SimpleExporterInput(documento));
exportador.setExporterOutput(new SimpleWriterExporterOutput(rutaRtf, "UTF-8"));
exportador.exportReport();
```

La fuente original trasladaba la codificación a una configuración RTF. La implementación corregida la aplica al writer. El E2E verifica la cabecera RTF del archivo. Como reto acumulativo, el punto añade `net.sf.jasperreports.engine.export.oasis.JR0dtExporter` para producir `output/informe_ventas.odt` a partir del mismo JasperPrint.

Bloque 4 — Un JasperPrint, varios formatos

Al llegar a 6.4 una sola ejecución de `GeneradorInformeVentas` produce PDF, PDF protegido, XLSX, HTML, CSV, XML, RTF y ODT. Todos parten del mismo objeto documento.

TEXT



Esta arquitectura es más eficiente que volver a llenar el informe para cada formato. Los parámetros, consulta y totales son idénticos para todas las salidas. Si una exportación lanza excepción, el catch final registra la traza y `System.exit(1)` hace visible el fallo en CI.

Bloque 5 — Contratos de archivo y E2E multiformato

Cada formato necesita una evidencia distinta. El workflow aplica contratos estructurales: PDF con firma PDF, XLSX como ZIP OOXML y hojas Ventas/Catálogo, HTML con etiquetas y CSS, CSV con BOM/delimitador, XML con declaración XML, RTF con su firma y ODT como paquete OpenDocument válido.

TEXT

Formato	Evidencia mínima
PDF	firma PDF + pdfinfo
XLSX	PK + ZIP íntegro + workbook.xml
HTML	charset + título + CSS + cierre
CSV	BOM UTF-8 + ; + registros
XML	declaración XML
RTF	cabecera RTF
ODT	ZIP íntegro + mimetype OpenDocument Text

A la vez, el workflow comprueba SQLite y las seis páginas de `informe_ventas`. `EXPORTACION_OTROS.md` documenta las decisiones corregidas y el JRXML/JRTX continúan byte a byte iguales a M5/5.6.

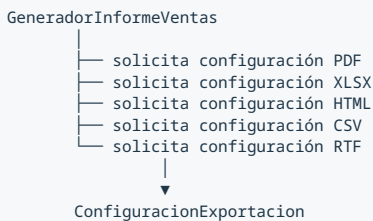
Punto 6.5 — Configuración de exportación

Objetivos de aprendizaje

- Comprender el papel del objeto `JasperReportsContext` en la configuración global.
- Utilizar las propiedades del sistema para configurar el comportamiento de los exportadores.
- Crear un archivo `jasperreports.properties` con propiedades por defecto.
- Definir una clase de configuración personalizada para centralizar las opciones del proyecto.
- Combinar la configuración global con la configuración por exportador.
- Documentar la configuración de exportación del proyecto `EditorialReports`.

Bloque 1 — Por qué centralizar la configuración

Después de cuatro puntos, `GeneradorInformeVentas` conoce todos los exportadores y contiene muchas opciones específicas. El objetivo de 6.5 es separar políticas reutilizables de la lógica de orquestación. `ConfiguracionExportacion.java` encapsula la creación de las configuraciones; `GeneradorInformeVentas` conserva rutas, `JasperPrint` y orden de exportación.



TEXT

Esta refactorización no cambia el JRXML ni las salidas esperadas. El E2E vuelve a generar todos los formatos para demostrar que la centralización no introduce regresiones.

Bloque 2 — ReportConfiguration frente a ExporterConfiguration

XLSX muestra por qué una clase central no debe ocultar las categorías de configuración. Las opciones de una hoja concreta pertenecen a `SimpleXlsxReportConfiguration`; las del libro/exportador pertenecen a `SimpleXlsxExporterConfiguration`. Por eso la clase ofrece dos métodos.

```

public static SimpleXlsxReportConfiguration getConfiguracionXlsxReport(String nombreHoja)
public static SimpleXlsxExporterConfiguration getConfiguracionXlsxExportador()
  
```

JAVA

El generador aplica ambas con `setConfiguration`. Esta separación conserva el modelo de la API y hace explícito qué propiedades pueden depender del informe. Es una corrección frente al origen, que trataba todas las opciones como una única configuración.

PDF, HTML y CSV tienen sus propios tipos de configuración. RTF usa una configuración vacía en el checkpoint porque la codificación se sigue definiendo en `SimpleWriterExporterOutput`.

Bloque 3 — jasperreports.properties y classpath

JasperReports puede leer propiedades globales desde `jasperreports.properties` cuando el archivo está disponible en el classpath. El checkpoint crea el archivo dentro de `EditorialReportsJava/src` y modifica Maven para copiar los recursos no Java de esa carpeta a `target/classes`.

```

net.sf.jasperreports.export.pdf.compressed=true
net.sf.jasperreports.export.csv.field.delimiter=;
  
```

PROPERTIES

El E2E exige que `target/classes/jasperreports.properties` exista después de `mvn package`. Así se prueba la premisa de la explicación: no basta con crear el fichero en el árbol fuente si el runtime nunca puede cargarlo.

Las propiedades globales ofrecen valores predeterminados; la configuración específica aplicada a un exportador puede concretar o sobrescribir comportamiento para una operación determinada.

Bloque 4 — Métodos de fábrica del proyecto

`ConfiguracionExportacion` actúa como una fábrica estática sencilla. Los métodos reciben únicamente los datos variables —por ejemplo título, autor o nombre de hoja— y fijan las convenciones del proyecto.

JAVA

```
SimplePdfExporterConfiguration c =
    ConfiguracionExportacion.getConfiguracionPdf(
        "Informe de Ventas - EditorialReports",
        "Departamento Comercial"
    );
```

El PDF centraliza metadatos y compresión. XLSX centraliza opciones de hoja y libro. HTML centraliza cabecera, pie y separador. CSV centraliza delimitadores y BOM. RTF devuelve la configuración del exportador mientras la salida conserva UTF-8.

El objetivo no es crear una abstracción universal sobre JasperReports, sino reducir duplicación manteniendo visibles los tipos reales de la biblioteca.

Bloque 5 — Cierre técnico y trazabilidad del módulo

El checkpoint 6.5 añade tres elementos físicos: CONFIGURACION_EXPORTACION.md, ConfiguracionExportacion.java y jasperreports.properties; modifica GeneradorInformeVentas.java y pom.xml. Ningún archivo heredado se elimina.

TEXT

```
M5/5.6
├─ 6.1 PDF
│   └─ 6.2 XLSX
│       └─ 6.3 HTML
│           └─ 6.4 CSV/XML/RTF
│               └─ 6.5 configuración central
```

La validación automática busca las llamadas a los métodos centrales, comprueba que el properties llega al classpath y vuelve a validar todos los formatos. Los invariantes siguen siendo 14 libros, 9 ventas, 31 unidades y 633,40 €, con seis páginas en el informe de ventas.

Con este punto, la fuente original del M6 queda implementada en una forma compatible con JasperReports 6.20.0: se preservan los objetivos —exportar y centralizar configuración— y se corrigen las APIs que no compilaban tal como estaban escritas.